

App Rebuild Blueprint Document

What & Why

The user wants to rebuild this Wegen NFT Trait Store on a different platform, keeping the same concept, user flows, features, pages, and functionality. They need a single, extremely detailed rebuild blueprint document that fully documents the current app so another AI coding tool (with no access to this Replit project) can recreate it from scratch. This is a documentation/analysis deliverable, not a code change to the running app.

Done looks like

- A single well-organized Markdown file (e.g. `rebuild-blueprint.md` at the project root, or wherever the executor deems appropriate) exists containing all of the following sections, each filled out with real, specific detail pulled from the actual codebase (not generic placeholders):
- Plain-English summary of what the app does, who it's for, what problem it solves.
- Complete feature list — every page, dashboard, form, button, user action, admin action, DB action, upload, API call, automation, notification, payment/purchase flow, login/wallet-connect flow, and background process.
- Page-by-page breakdown for every route: URL, purpose, user actions available, buttons/actions, data shown, data created/edited/deleted, and what happens after each action.
- User-flow breakdown per role (new visitor, wallet-connected user, admin, buyer/seller in swap & market, bounty participant, etc.) describing the full journey end-to-end.
- Database/schema breakdown: every table, every field, relationships, example records, and what data each feature needs persisted.
- Backend/API breakdown: every route/endpoint, method, input, output, and which frontend feature calls it.

- Environment variables & outside services: every external dependency (DB, wallet/auth flow, object storage, price-feed/ETH conversion, etc.) with placeholder env var names only — no real secret values.
- Design & UI notes: visual style, layout patterns, color/theme system, component conventions, modals, cards, admin dashboard layout, mobile behavior.
- Current issues / broken or incomplete parts, each with: where it happens, what's wrong, what correct behavior should be.
- Fresh build plan broken into phases (Foundation, Database, Authentication, Core pages, Core features, Admin tools, Testing, Launch polish).
- A copy-ready prompt for another AI builder tool, written so it can rebuild the app from scratch on another platform using this blueprint, assuming zero visibility into this Replit project.
- The document is presented to the user as a downloadable/viewable file when finished.

Out of scope

- No changes to the running application code, database, or configuration.
- No actual migration or rebuild work — this task only produces the planning/documentation artifact.
- Do not guess at broken/incomplete parts; verify by reading code, running typecheck, or checking logs before listing something as an issue.

Steps

1. **Inventory the frontend** — Walk every route/page in the store frontend, documenting purpose, on-page actions, buttons, and data displayed/mutated for each (Store, Locker, My Wegens/NFTs, Swap, Sandbox, My Legends, Bounties, Packs & Points, Admin).
2. **Inventory the backend & API contract** — Walk the OpenAPI spec and route handlers to document every endpoint: method, path, inputs, outputs, and which frontend page/action triggers it (auth/nonce/session, traits, locker, nfts, admin, market, swap, storage, points, bundles).
3. **Inventory the database schema** — Document every table, column, type, and relationship, plus give representative example records and note which features write/read each table.

4. **Document wallet & auth flow** — Explain the SIWE (Sign-In With Ethereum) challenge/response flow, supported wallets, and how sessions gate the Locker/NFT/Admin pages.
5. **Document external dependencies & env vars** — List services this app depends on (Postgres, object storage, ETH price-conversion source, wallet providers) with placeholder env var names, no secret values.
6. **Document design/UI system** — Capture the visual style, layout conventions, component library usage, rarity badges, theming, and admin dashboard layout.
7. **Identify current issues** — Review for malfunctioning, incomplete, duplicated, insecure, or confusing behavior (e.g. admin panel has no auth by default, NFT ownership isn't read on-chain) and any other issues found via code inspection or logs.
8. **Write the fresh-build phase plan** — Order rebuild phases (Foundation → Database → Auth → Core pages → Core features → Admin tools → Testing → Launch polish) referencing the concepts documented above.
9. **Write the copy-ready AI-builder prompt** — Synthesize all of the above into a single detailed prompt a user can paste into another AI coding tool, assuming that tool has no access to this project.
10. **Assemble and present** — Compile all sections into one Markdown document and present it to the user as a file for download/review.

Relevant files

- `artifacts/wegen-trait-store/src/App.tsx`
- `artifacts/wegen-trait-store/src/contexts/WalletContext.tsx`
- `artifacts/wegen-trait-store/src/pages/Admin.tsx`
- `artifacts/api-server/src/routes/auth.ts`
- `artifacts/api-server/src/routes/traits.ts`
- `artifacts/api-server/src/routes/locker.ts`
- `artifacts/api-server/src/routes/nfts.ts`
- `artifacts/api-server/src/routes/admin.ts`
- `artifacts/api-server/src/routes/market.ts`
- `artifacts/api-server/src/routes/swap.ts`
- `artifacts/api-server/src/routes/storage.ts`
- `artifacts/api-server/src/routes/points.ts`

- `artifacts/api-server/src/routes/bundles.ts`
- `lib/db/src/schema`
- `lib/api-spec/openapi.yaml`
- `lib/api-spec/orval.config.ts`
- `lib/api-zod`
- `replit.md`